

Act as a Principal Decision Intelligence Architect, AI Systems Architect, Simulation Engineer, Business Strategist, Risk Analyst, Financial Analyst, JavaScript Developer, SaaS Security Engineer, and Product Designer.

Your task is to build the final intelligence layer of the INFINICUS AI Business Decision Intelligence Platform as a standalone HTML application.

The layer is called:

AI DECISION INTELLIGENCE LAYER

This layer depends on:

1. Data Acquisition Layer
2. Business Operations Layer
3. Business Intelligence Layer
4. Business Digital Twin Layer
5. Existing Simulation Engine

Do not rewrite or duplicate any completed lower layer.

Core Purpose

Convert business goals, Digital Twin state, simulation results, constraints, risks, opportunities, and historical outcomes into structured, ranked, explainable, and auditable business recommendations.

The layer must answer:

1. What decision should the business consider?
2. Why should it consider that decision?
3. What evidence supports it?
4. What outcome is expected?
5. What risks could prevent success?
6. What alternatives exist?
7. How confident is the platform?
8. What implementation steps are required?
9. Did the decision succeed after execution?

The platform must remain advisory.

Important decisions require authorized human approval.

Decision Processing Flow

Business goal or decision trigger → Decision problem framing → Candidate action generation → Policy validation → Digital Twin scenario creation → Simulation → Feasibility evaluation → Risk evaluation →

Opportunity evaluation → Financial and operational impact evaluation → Multi-criteria scoring → Decision ranking → Explainable recommendation → Human approval → Implementation plan → Operations handoff → Outcome monitoring → Actual-versus-predicted reconciliation → Controlled calibration

Existing Architecture Compatibility

Use:

```
window.INFINICUS.bus.register(route, handler, schema)
```

```
window.INFINICUS.bus.call(route, payload)
```

```
window.INFINICUS.bus.emit(event, data)
```

```
window.INFINICUS.bus.on(event, listener)
```

Use the namespace:

```
decision.*
```

Do not create another event bus.

Required Dependencies

Consume:

- Digital Twin state
- Twin version
- Twin fidelity score
- Twin constraints
- Twin dependencies
- Twin assumptions
- Simulation-ready context
- Scenario simulation results
- Business goals
- Business policies
- Historical decisions
- Actual outcomes

Expected Digital Twin routes include:

```
twin.state.get twin.fidelity.get twin.constraint.list twin.dependency.list twin.assumption.list  
twin.scenario.create twin.context.build
```

Use the existing Simulation Engine through its registered EngineBus routes.

Do not recreate Monte Carlo logic.

Required Modules

Build:

1. Decision Workspace
2. Goal and Objective Registry
3. Decision Trigger Engine
4. Decision Problem Framing
5. Candidate Action Generator
6. Decision Policy Registry
7. Simulation Orchestrator
8. Feasibility Engine
9. Risk Intelligence Engine
10. Opportunity Intelligence Engine
11. Financial Impact Engine
12. Operational Impact Engine
13. Multi-Criteria Decision Engine
14. Decision Ranking Engine
15. Recommendation Engine
16. Alternative Strategy Engine
17. Explanation and Evidence Engine
18. Decision Confidence Engine
19. Human Approval Workflow
20. Decision Plan Generator
21. Execution Handoff
22. Decision Memory System
23. Outcome Monitoring
24. Prediction Reconciliation
25. Learning and Calibration Engine
26. Decision dashboards
27. Executive Briefing Generator

Decision Trigger Sources

Support:

user request missed target anomaly negative trend constraint opportunity Business Health decline Digital
Twin update cash risk inventory risk customer churn supplier disruption strategic milestone simulation
result

Problem Framing

Every problem must identify:

problem statement business objective affected metrics affected entities constraints dependencies urgency
decision boundary data confidence

Distinguish:

symptom root problem contributing factor constraint objective assumption

Candidate Action Generation

Generate multiple candidate actions.

Each candidate must include:

candidateId action type title description affected entities changed variables implementation requirements
source status

Candidates must pass policy, authority, legal, regulatory, financial, and operational validation before simulation.

Decision Policy Registry

Support policies for:

pricing hiring termination procurement inventory marketing capital expenditure borrowing branch
expansion customer credit supplier change cash management data privacy

Support:

platform policies industry policies country policies business policies user authority limits

Simulation Orchestrator

For each approved candidate:

1. Create an isolated Digital Twin scenario branch.
2. Apply candidate changes.
3. Build a simulation context.
4. Run the existing Simulation Engine.
5. Validate the result.
6. Store the complete simulation evidence.
7. Return normalized results to the Decision Engine.

Every simulation must preserve:

candidateId twinId twinVersion simulationContextId simulation engine version assumption versions
benchmark versions run timestamp

Feasibility Evaluation

Evaluate:

financial operational technical workforce supplier regulatory market time data authority

Return:

overall score dimension scores blockers conditions confidence status

Risk Intelligence

Evaluate:

financial risk cash-flow risk market risk customer risk operational risk inventory risk supplier risk workforce
risk legal risk regulatory risk technology risk reputation risk execution risk data-quality risk

Every risk must include:

probability impact exposure score severity evidence mitigation confidence

Opportunity Intelligence

Evaluate:

revenue opportunity profit opportunity cash-flow opportunity customer-growth opportunity retention
opportunity market-share opportunity efficiency opportunity cost-reduction opportunity capacity
opportunity supplier opportunity expansion opportunity risk-reduction opportunity

Impact Analysis

Generate:

implementation cost revenue change expense change profit change cash-flow change payback period ROI
operational workload training requirements implementation time affected business areas

Always provide:

downside case expected case upside case

Never present a predicted value as guaranteed.

Multi-Criteria Decision Engine

Rank candidates using transparent, versioned criteria.

Evaluate:

financial return risk feasibility strategic alignment customer impact cash impact operational complexity
time to value reversibility data confidence

Allow weights to vary by objective.

Examples:

cash crisis growth profit improvement cost reduction risk reduction startup validation expansion

Recommendation Structure

Every final recommendation must contain:

decisionId businessId twinId twinVersion problem goal recommended action reasoning expected outcome
downside case expected case upside case risks mitigations alternatives implementation conditions required
approval confidence evidence generatedAt

Explanation and Evidence

Label all evidence as one of:

observed fact calculated metric simulation prediction estimated assumption external benchmark AI
inference

Do not invent evidence.

Do not present correlation as confirmed causation.

Clearly state uncertainty and missing information.

Human Approval

Require approval for:

hiring and termination loans and borrowing branch opening or closure large capital expenditure significant
price changes major supplier changes customer-credit changes legal actions regulatory actions data-
sharing changes

Support:

pending approved rejected revision requested expired cancelled

Decision Plan

After approval, generate:

objective implementation steps owners deadlines success metrics monitoring period risk controls rollback conditions

Execution Handoff

Do not directly edit operational records.

Send approved actions through Business Operations routes such as:

operations.task.create operations.product.update operations.purchaseOrder.create
operations.inventory.adjust operations.employee.create

Only use routes permitted by policy and approval status.

Decision Memory

Preserve:

problem candidate actions rejected alternatives simulation results risks opportunities recommendation
evidence confidence approval history implementation actual outcome prediction error calibration changes

Outcome Monitoring

Monitor:

success metrics baseline values target values current values risk indicators rollback conditions
implementation status

Support:

not started on track at risk off track completed stopped

Reconciliation

Compare:

predicted result actual result prediction error evaluation window data confidence outcome status

Use outcome statuses:

successful partially successful unsuccessful inconclusive cancelled

Controlled Calibration

Use decision outcomes to propose changes to:

simulation assumptions risk probabilities ranking weights industry benchmarks confidence penalties
implementation-delay estimates

Do not permit uncontrolled self-modification.

Every calibration must require:

minimum evidence threshold validation versioning audit logging rollback capability approval where
required

Security and Governance

Implement:

business ownership validation role-based access approval enforcement authority limits policy validation
scenario isolation safe HTML rendering input sanitization audit logs duplicate decision prevention
recommendation versioning simulation-result traceability confidence validation calibration rollback cross-
business isolation

Do not make autonomous employment, credit, legal, or regulatory decisions.

Do not use sensitive personal attributes for decision ranking.

User Interface

Use:

black background neon green accents white typography IBM Plex Sans IBM Plex Mono mobile-first
responsive design desktop left navigation clear priority labels visible approval status confidence indicators
risk severity scenario comparison evidence viewer decision history monitoring dashboards

Desktop navigation:

Decision Center Goals Problems Recommendations Scenarios Approvals Implementation Monitoring
Decision History Learning Policies Settings

Mobile navigation:

Decisions Recommendations Approvals Monitoring More

Storage

Use:

localStorage for preferences IndexedDB for decisions, evidence, simulations, approvals, monitoring, and history a clean adapter for future Firestore, Supabase, or another cloud database

Use separate stores for:

decision workspaces goals triggers problems candidate actions policies simulation orchestrations feasibility results risks opportunities impact results criteria profiles rankings recommendations alternatives explanations confidence scores approvals decision plans execution handoffs decision memory monitoring reconciliations calibration proposals briefings audit logs

Do not store the entire layer inside one unstructured JSON record.

Build Method

Build in small executable blocks.

For each block:

1. State its purpose.
2. State dependencies.
3. Provide only the required code.
4. Explain exactly where to insert it.
5. Provide a manual test procedure.
6. Review security, governance, accuracy, and data-integrity risks.
7. Stop after completing the block.

Do not regenerate the full application after every block.

Do not remove or rewrite previously working modules.

First Response

Do not begin coding immediately.

First provide:

1. Decision Intelligence architecture.
2. Module dependency map.
3. Goal and trigger model.
4. Problem-framing schema.

5. Candidate-action schema.
6. Policy model.
7. Simulation orchestration contract.
8. Feasibility and risk methods.
9. Multi-criteria scoring method.
10. Recommendation schema.
11. Approval workflow.
12. Decision Memory design.
13. Outcome-monitoring design.
14. Calibration controls.
15. EngineBus route map.
16. Event map.
17. Storage architecture.
18. Exact development sequence.

After the architecture review, begin with Block 1 only.